

MERKO

Engineering Handbook

v3.0 — Single Source of Truth

Enterprise Customizable Product Marketplace

Attribute	Value
Version	3.0.0
Status	Authoritative — Reconstructed
Stack	Next.js 15 · Express.js · PostgreSQL · Prisma · Redis · Cloudinary · Razorpay
Phases	12
Sprints	24 (1-week each, ~6 months)
Total API Endpoints	62
Total DB Tables	22
Total Features	89
Classification	Internal Technical Reference
Last Updated	June 2026

1. Executive Summary

Merko is a Flipkart-style customizable product marketplace enabling customers to browse, personalize, preview, and purchase printed and customized physical goods — ID cards, mugs, T-shirts, banners, and more.

The platform is built around a dynamic, admin-configurable customization engine that allows product managers to define custom fields, input types, and preview logic without any code changes. Admin and SuperAdmin roles have full product lifecycle control including creation, editing, activation/deactivation, and deletion — all surfaced in the Management Portal.

1.1 Product Vision

"Make professional customized printing accessible to everyone — browse, personalize, preview, and order in under 5 minutes."

1.2 Problem Statement

Problem	Impact
Local printing businesses have no digital storefront	Lost revenue, no reach beyond walk-ins
Customers cannot preview customized products before ordering	High return rates, low conversion
Admin teams must involve developers to add new product fields	Slow time-to-market, high operational cost
No centralized order + production tracking for print businesses	Fulfillment errors, poor customer experience
No admin-controlled product visibility or lifecycle management	Stale products surface, catalog integrity lost

1.3 Success Metrics

Metric	Target (6 months)
Registered users	10,000+
Monthly active orders	2,000+
Average order value	₹350+
Cart-to-checkout conversion	≥ 35%
Page load time (LCP)	< 2.5s
Uptime	≥ 99.9%
Admin field setup time	< 10 minutes (no code)
Admin product creation time	< 5 minutes

1.4 Scope — v1 MVP

In Scope

- Product catalog with categories and search
- Dynamic customization engine (admin-configured fields, no code)
- Live design preview for customizable products
- File upload (photos, logos, designs) via Cloudinary
- Cart and multi-product checkout
- Razorpay payment integration (UPI, Card, NetBanking, Wallets)
- Order tracking with full status lifecycle
- Customer notifications (Email + WhatsApp + Push + In-App)
- Admin Management Portal: full product CRUD, field builder, order pipeline, coupons, analytics
- Product visibility controls (activate/deactivate/delete/restore) — admin only
- JWT authentication with refresh tokens and RBAC

Out of Scope (v2+)

- Vendor multi-seller portal
- AI design suggestion engine
- Mobile native app (iOS/Android)
- Loyalty/referral programs
- Bulk B2B order portal

2. User Roles & Permissions

Merko has four roles with strictly enforced RBAC middleware on every protected endpoint.

2.1 Role Matrix

Capability	Guest	Customer	Admin	SuperAdmin
Browse products	✓	✓	✓	✓
Search & filter	✓	✓	✓	✓
Customize product (preview)	✓	✓	✓	✓
Add to cart / Checkout	✗	✓	✗	✗
Track own orders	✗	✓	✓	✓
Write reviews	✗	✓	✗	✗
Create / Edit products	✗	✗	✓	✓
Activate / Deactivate products	✗	✗	✓	✓
Delete / Restore products	✗	✗	✓	✓
Manage customization fields	✗	✗	✓	✓
Manage product images & templates	✗	✗	✓	✓
Manage product variants & pricing	✗	✗	✓	✓
Manage orders (status pipeline)	✗	✗	✓	✓
Manage coupons	✗	✗	✓	✓
View analytics & reports	✗	✗	✓	✓
Manage admin accounts	✗	✗	✗	✓
System configuration	✗	✗	✗	✓
Broadcast notifications	✗	✗	✗	✓
Export customer data	✗	✗	✗	✓

2.2 Role Definitions

Guest

Unauthenticated visitor. Can browse, search, preview customizations. Cannot persist cart or complete purchase. Prompted to register at cart/checkout.

Customer

Registered, verified user. Full shopping journey. Can manage saved addresses, order history, saved designs, and submit post-delivery reviews.

Admin

Operations team member. Full product lifecycle management (create/edit/activate/deactivate/delete/restore products). Manages customization field builder, order pipeline, gallery, coupons, and reviews. Cannot manage other admin accounts.

SuperAdmin

Full system access. Manages admin accounts and roles. Access to system configuration, raw analytics, audit logs, and customer data export. Can impersonate users for support (logged).

3. Product Management — Complete Specification

Product management is a core admin capability. Every product action is performed through the Management Portal (/admin/products/*) and surfaced to customers on the Customer Portal. No developer intervention is required for any product lifecycle action.

3.1 Product Lifecycle States

State	isActive	Visible to Customers	Trigger
DRAFT	false	No	Admin creates product, not yet published
ACTIVE	true	Yes	Admin publishes / activates product
INACTIVE	false	No	Admin deactivates (soft hide, stock preserved)
DELETED	false	No	Admin soft-deletes (recoverable, archived)
RESTORED	true	Yes	Admin restores from deleted state

3.2 Admin Product Capabilities

- Create product — name, description, base price, SKU, category, tags, images, variants, customization fields
- Edit product — update any field at any time; cache invalidated automatically on save
- Upload product images — up to 10 images per product via Cloudinary CDN
- Upload printable artwork / templates — design_templates table, used by Preview Engine
- Add / remove / edit product variants — size, color, material with individual pricing and stock
- Manage pricing — base price in paise, per-variant price delta, no rounding errors
- Manage inventory — per-variant stock count; low-stock warnings in admin dashboard
- Manage product categories — hierarchical parent/child category assignment
- Configure customization options — no-code field builder (9 field types)
- Activate product — sets isActive=true, product appears on storefront
- Deactivate product — sets isActive=false, product hidden from storefront, stock preserved
- Soft delete product — archived, hidden, recoverable within 30 days
- Restore product — reverses soft delete, product re-enters INACTIVE state
- Bulk product import — CSV upload (v2 scope noted, schema ready)
- Bulk product export — CSV/PDF of full catalog
- Product SEO metadata — slug (auto-generated), tags array stored in DB
- Product approval workflow — DRAFT → ACTIVE gating by SuperAdmin (configurable)

3.3 Product Management API Endpoints

Method	Endpoint	Role	Description
GET	/api/v1/products	Public	Paginated listing, filters, sort

GET	/api/v1/products/search?q=	Public	Full-text search (GIN index)
GET	/api/v1/products/:slug	Public	Product detail by slug
GET	/api/v1/products/:id/schema	Public	Customization field schema
POST	/api/v1/products	Admin	Create new product
PUT	/api/v1/products/:id	Admin	Update product (all fields)
PATCH	/api/v1/products/:id/status	Admin	Activate / Deactivate
DELETE	/api/v1/products/:id	Admin	Soft delete product
POST	/api/v1/products/:id/restore	Admin	Restore soft-deleted product
PUT	/api/v1/products/:id/schema	Admin	Update customization schema
POST	/api/v1/products/:id/variants	Admin	Add variant
PUT	/api/v1/products/:id/variants/:vid	Admin	Edit variant
DELETE	/api/v1/products/:id/variants/:vid	Admin	Delete variant
POST	/api/v1/products/:id/images	Admin	Upload product image
DELETE	/api/v1/products/:id/images/:imgId	Admin	Remove product image
POST	/api/v1/products/:id/templates	Admin	Upload design template
GET	/api/v1/categories	Public	Full category tree
POST	/api/v1/categories	Admin	Create category
PUT	/api/v1/categories/:id	Admin	Update category
DELETE	/api/v1/categories/:id	Admin	Delete category

3.4 Admin Management Portal — Product Pages

- /admin/products — sortable/filterable data table with status badges, quick-activate toggle, bulk actions
- /admin/products/new — full product creation form: basic info → images → variants → customization fields → preview template
- /admin/products/[id] — edit all product fields; status control (Activate / Deactivate / Delete / Restore) in sidebar
- /admin/products/[id]/fields — no-code customization field builder (drag-drop, 9 field types)
- /admin/products/[id]/templates — design template canvas config (position, font, overlay)
- /admin/products/[id]/inventory — per-variant stock management with low-stock threshold alerts
- /admin/categories — category CRUD with parent/child hierarchy and sort order

4. Complete Feature Inventory

4.1 Authentication Features (AUTH)

ID	Feature	Priorty
AUTH-01	User registration — name, email, phone, password	P0
AUTH-02	Phone OTP verification on registration (6-digit, 5-min TTL)	P0
AUTH-03	Login — email + password → JWT in httpOnly cookie	P0
AUTH-04	JWT access token (15-min) + refresh token (7-day)	P0
AUTH-05	Refresh token rotation — invalidate old on every use	P0
AUTH-06	Forgot password via email OTP	P1
AUTH-07	Google OAuth SSO	P2
AUTH-08	Admin login with TOTP 2FA	P1
AUTH-09	Session revocation from admin panel	P1
AUTH-10	Rate limiting: 5 req/min on /auth/* endpoints	P0
AUTH-11	OTP lockout after 3 failed attempts (15-min Redis lock)	P0

4.2 Product & Catalog Features (PROD)

ID	Feature	Priorty
PROD-01	Product listing with pagination (20/page)	P0
PROD-02	Category hierarchy (parent/child) with image and sort order	P0
PROD-03	Full-text product search (GIN index on name + description)	P0
PROD-04	Filter by: category, price range, rating, availability	P0
PROD-05	Sort by: price asc/desc, newest, best-selling	P0

PROD-06	Product detail page with image gallery (lightbox)	P0
PROD-07	Product variants (size, color, material) — individual price/stock	P0
PROD-08	Related products section	P1
PROD-09	Recently viewed products (local + DB)	P1
PROD-10	Real-time stock status per variant	P0
PROD-11	Product creation — admin, no-code	P0
PROD-12	Product editing — all fields, any time	P0
PROD-13	Product activation / deactivation	P0
PROD-14	Product soft delete and restore	P0
PROD-15	Product image upload via Cloudinary (up to 10/product)	P0
PROD-16	Design template upload and config per product	P0
PROD-17	Product SEO slug (auto-generated, editable)	P0
PROD-18	Product tags array for cross-linking	P1
PROD-19	Bulk product export (CSV)	P1
PROD-20	Low-stock alerts in admin dashboard	P1

4.3 Customization Engine Features (CUST)

ID	Feature	Priority
CUST-01	Admin defines custom fields per product — no code	P0
CUST-02	9 field types: text, textarea, number, image, dropdown, checkbox, radio, color picker, dimension	P0
CUST-03	Per-field validation: required, min/max length, format, file type	P0
CUST-04	Drag-and-drop field ordering by admin	P1

CUST-05	Live preview canvas updates on every field change ($\leq 200\text{ms}$)	P0
CUST-06	Preview template: define text position, font, image area per product	P0
CUST-07	Save design to user account for reuse	P1
CUST-08	Customization data serialized as JSONB and stored with order item	P0
CUST-09	Admin previews customer's customization data from order panel	P0
CUST-10	Print-ready file generation (PNG/PDF) via Puppeteer/Sharp	P1

4.4 Cart & Order Features (CART / ORD)

ID	Feature	Priority
CART-01	Persistent cart — DB-backed for authenticated users	P0
CART-02	Guest cart — localStorage, migrate on login	P1
CART-03	Add / update quantity / remove cart items	P0
CART-04	Cart item stores variant + customization snapshot	P0
CART-05	Dynamic pricing: base + variant delta + customization surcharge	P0
CART-06	Coupon code application with live discount calculation	P0
ORD-01	Atomic order creation (cart → order + items in one transaction)	P0
ORD-02	Order status lifecycle: PENDING → PAYMENT_CONFIRMED → IN_PRODUCTION → QUALITY_CHECK → SHIPPED → DELIVERED	P0
ORD-03	Order cancellation (before IN_PRODUCTION) with auto-refund	P1
ORD-04	Order history with filters and pagination	P0
ORD-05	Re-order functionality	P2
ORD-06	Tracking number + courier stored on SHIPPED update	P0
ORD-	Print-ready file download per order item (admin)	P1

07		
----	--	--

4.5 Payment Features (PAY)

ID	Feature	Priority
PAY-01	Razorpay checkout: UPI, card, netbanking, wallets	P0
PAY-02	Webhook signature verification (HMAC-SHA256 on raw body)	P0
PAY-03	Idempotency key on all payment creation requests	P0
PAY-04	Refund initiation from admin panel	P1
PAY-05	Payment failure retry (max 3 attempts before abandonment)	P0
PAY-06	Invoice PDF auto-generation and email attachment	P1
PAY-07	Stock decrement with row-level lock on capture only (prevent oversell)	P0
PAY-08	Duplicate webhook idempotency (200 no-op on replay)	P0

4.6 Admin & Notification Features

ID	Feature	Priority
ADM-01	Admin dashboard: revenue, orders, top products KPIs	P0
ADM-02	Product CRUD with image upload and variant management	P0
ADM-03	Dynamic field builder per product (no-code)	P0
ADM-04	Order management with full status pipeline	P0
ADM-05	Customer list with order history and account status	P1
ADM-06	Coupon CRUD: percentage, flat, free-shipping	P1
ADM-07	Gallery management (portfolio/completed work)	P1
ADM-08	Review moderation (approve/reject/reply)	P1
ADM-09	Export orders and revenue as CSV/PDF	P1
ADM-10	Notification broadcast to all customers	P2
NOTIF-01	Email notifications via SendGrid (HTML templates)	P0

NOTIF-02	WhatsApp notifications via Twilio Business API	P0
NOTIF-03	Push notifications via Firebase FCM	P1
NOTIF-04	In-app notification feed (bell icon + badge)	P1
NOTIF-05	Bull MQ queues with retry + dead-letter handling	P0

5. High-Level Architecture

Layer	Technology	Purpose
Client Tier	Next.js 15 (App Router)	Customer Portal + Admin Management Portal — SSR/ISR
Edge & CDN	Cloudflare WAF + Vercel Edge	DDoS protection, CDN, SSR, ISR caching
API Tier	Express.js on Railway	RESTful API Gateway — 8 service modules
Background Jobs	Bull MQ + Redis (Upstash)	Async notifications, print jobs, refund processing
Primary Database	PostgreSQL on Railway	All transactional data — 22 tables
Read Replica	PostgreSQL Read Replica	Admin analytics and reporting queries
Cache	Redis (Upstash)	Session store, rate limiting, product cache, queues
File Storage	Cloudinary	Product images, user uploads, preview renders, templates
Payments	Razorpay	UPI, card, netbanking, wallets — PCI-DSS SAQ A
Email	SendGrid	Transactional email with HTML templates
WhatsApp	Twilio Business API	Approved template messages
Push	Firebase FCM	Mobile and web push notifications
Monitoring	Sentry + Railway Metrics	Error tracking, performance, alerting
Connection Pool	PgBouncer	100 PostgreSQL connections serving 1000+ app clients

5.1 Dual-Portal Architecture

Merko operates two distinct Next.js 15 portals sharing a single backend API:

- Customer Portal (merko.in) — public storefront: browse, customize, cart, checkout, order tracking, profile
- Admin Management Portal (merko.in/admin/*) — protected by ADMIN/SUPER_ADMIN role: all product lifecycle actions, order pipeline, analytics, field builder

Both portals are deployed on Vercel and share the same Express.js API on Railway. RBAC middleware enforces access at the API layer — the frontend is the UI; the API is the authority.

5.2 API Module Breakdown

Module	Responsibility	Key Tables
Auth Module	Registration, OTP, JWT, RBAC, session	users, activity_logs
Product Module	Full product lifecycle, categories,	products, product_variants, categories,

	variants, images	product_images
Customization Module	Field schema CRUD, design saves, upload	design_templates, user_designs
Cart + Order Module	Cart persistence, checkout, order lifecycle	carts, cart_items, orders, order_items, addresses, coupons
Payment Module	Razorpay integration, webhook, refunds	payments
Notification Module	Bull MQ orchestration, multi-channel delivery	notifications
Admin Module	Dashboard KPIs, analytics, CSV export, gallery, reviews	activity_logs, gallery_images, reviews
Upload Module	Cloudinary signed upload, MIME validation, cleanup	All tables with image/file URLs

6. Complete Database Schema — 22 Tables

All tables use UUID primary keys. All monetary values stored in paise (integer) to avoid floating-point errors. All timestamps are timestampz.

6.1 Table Inventory

#	Table	Purpose
1	users	All user accounts: customers, admins, superadmins
2	addresses	Customer delivery addresses (multiple per user)
3	categories	Product category hierarchy (self-referential parent/child)
4	products	Product master: name, price, SKU, customization schema (JSONB)
5	product_variants	Size/color/material variants with individual stock and price delta
6	product_images	Ordered image set per product (Cloudinary URLs)
7	design_templates	Canvas config (position, font, overlay) per product type
8	user_designs	Saved customer customization data (reusable)
9	carts	One active cart per authenticated user
10	cart_items	Items in cart: variant + customization snapshot + unit price
11	orders	Order header: totals, status, address, coupon reference
12	order_items	Line items: variant + immutable customization snapshot + previewUrl
13	payments	Razorpay order/payment/signature + status lifecycle
14	coupons	Discount codes: percentage, flat, free-shipping types
15	reviews	Customer product reviews (post-delivery, verified purchase)
16	notifications	In-app notification feed per user
17	activity_logs	Full audit trail of all admin mutations and user actions
18	gallery_images	Admin-managed portfolio images (completed work showcase)
19	user_sessions	Refresh token store (SHA-256 hash, userId, TTL)
20	otp_records	OTP store: phone/email, hash, expiry, attempt count
21	coupon_usages	Per-user coupon usage tracking (prevent abuse)

2 2	product_tags	Tag-to-product join table for cross-linking and search
--------	--------------	--

6.2 Critical Table Definitions

users

Column	Type	Constraints	Notes
id	UUID	PK, DEFAULT uuid()	
email	VARCHAR(255)	UNIQUE, NOT NULL	
phone	VARCHAR(15)	UNIQUE	Indian format validation
password_hash	TEXT	NOT NULL	bcrypt cost=12
name	VARCHAR(200)	NOT NULL	
avatar_url	TEXT		Cloudinary URL
role	UserRole enum	DEFAULT 'CUSTOMER'	CUSTOMER ADMIN SUPER_ADMIN
is_verified	BOOLEAN	DEFAULT false	Phone OTP gate
is_active	BOOLEAN	DEFAULT true	SuperAdmin can deactivate
fcm_token	TEXT		Firestore push token
created_at	TIMESTAMP PTZ	DEFAULT now()	
updated_at	TIMESTAMP PTZ	AUTO	

products

Column	Type	Constraints	Notes
id	UUID	PK	
category_id	UUID	FK → categories.id	
name	VARCHAR(500)	NOT NULL	
slug	VARCHAR(600)	UNIQUE, NOT NULL	Auto-generated, editable
description	TEXT	NOT NULL	
base_price_in_paise	INTEGER	NOT NULL	No floats — paise only

sku	VARCHAR(200)	UNIQUE, NOT NULL	
is_customizable	BOOLEAN	DEFAULT false	
customization_schema	JSONB		FieldDefinition[] array
images	JSONB	DEFAULT '[]'	Ordered Cloudinary URL array
tags	TEXT[]	DEFAULT '{}'	
is_active	BOOLEAN	DEFAULT false	Requires admin publish
is_deleted	BOOLEAN	DEFAULT false	Soft delete flag
deleted_at	TIMESTAMPZ		Set on soft delete
created_at	TIMESTAMPZ	DEFAULT now()	
updated_at	TIMESTAMPZ	AUTO	

orders

Column	Type	Constraints	Notes
id	UUID	PK	
order_number	VARCHAR(50)	UNIQUE, NOT NULL	Human-readable e.g. MRK-20260601-0001
user_id	UUID	FK → users.id	
address_id	UUID	FK → addresses.id	Snapshot at time of order
coupon_id	UUID	FK → coupons.id, NULL	
status	OrderStatus enum	DEFAULT 'PENDING'	Full lifecycle enum
subtotal_paise	INTEGER	NOT NULL	
discount_paise	INTEGER	DEFAULT 0	
shipping_paise	INTEGER	DEFAULT 0	
total_paise	INTEGER	NOT NULL	Verified server-side
tracking_number	VARCHAR(200)		Set on SHIPPED
courier	VARCHAR(200)		Set on SHIPPED

customer_notes	TEXT		
created_at	TIMESTAMP TZ	DEFAULT now()	
updated_at	TIMESTAMP TZ	AUTO	

6.3 Critical Indexes

```
CREATE INDEX idx_products_fts ON products USING gin(to_tsvector('english', name || ' ' || description));
CREATE INDEX idx_orders_status_created ON orders(status, created_at DESC);
CREATE UNIQUE INDEX idx_variant_sku ON product_variants(sku);
CREATE INDEX idx_variant_stock ON product_variants(product_id, stock) WHERE stock > 0;
CREATE UNIQUE INDEX idx_cart_item_unique ON cart_items(cart_id, variant_id);
CREATE INDEX idx_payments_captured ON payments(captured_at) WHERE status = 'CAPTURED';
CREATE INDEX idx_notifications_user_unread ON notifications(user_id, created_at DESC) WHERE is_read = false;
CREATE INDEX idx_products_active ON products(category_id, is_active) WHERE is_active = true AND is_deleted = false;
```

7. Complete API Inventory — 62 Endpoints

Base URL: /api/v1/ — All responses use envelope: { success, data, error, meta }

7.1 Authentication — /auth/*

Meth od	Endpoint	Auth	Description
POST	/auth/register	Public	Register with name, email, phone, password
POST	/auth/otp/send	Public	Send phone OTP (Twilio SMS)
POST	/auth/otp/verify	Public	Verify phone OTP → mark verified
POST	/auth/login	Public	Email + password → JWT cookies
POST	/auth/refresh	Cookie	Rotate refresh token → new access token
POST	/auth/logout	Cookie	Revoke refresh token from Redis
POST	/auth/forgot-password	Public	Send reset OTP to email
POST	/auth/reset-password	OTP token	Set new password
GET	/users/me	Customer	Get own profile
PATC H	/users/me	Customer	Update own profile

7.2 Products — /products/* and /categories/*

Meth od	Endpoint	Auth	Description
GET	/products	Public	Paginated list with filters (category, price, rating, sort)
GET	/products/search?q=	Public	Full-text search using PostgreSQL GIN index
GET	/products/:slug	Public	Product detail page data
GET	/products/:id/schema	Public	Customization field schema for product
POST	/products	Admin	Create new product (draft)
PUT	/products/:id	Admin	Update product fields
PATC H	/products/:id/status	Admin	Activate or Deactivate product
DELE TE	/products/:id	Admin	Soft delete product (recoverable)
POST	/products/:id/restore	Admin	Restore soft-deleted product
PUT	/products/:id/schema	Admin	Update customization field schema
POST	/products/:id/variants	Admin	Add product variant
PUT	/products/:id/variants/:vid	Admin	Edit variant (price, stock, options)

DELETE	/products/:id/variants/:vid	Admin	Remove variant
POST	/products/:id/images	Admin	Upload product image to Cloudinary
DELETE	/products/:id/images/:imgId	Admin	Remove product image
POST	/products/:id/templates	Admin	Upload design/print template
GET	/categories	Public	Full category tree
POST	/categories	Admin	Create category
PUT	/categories/:id	Admin	Update category
DELETE	/categories/:id	Admin	Delete category

7.3 Customization & Uploads — /designs/* /uploads/*

Method	Endpoint	Auth	Description
GET	/designs	Customer	List user's saved designs
POST	/designs	Customer	Save new design
GET	/designs/:id	Customer	Get single saved design
DELETE	/designs/:id	Customer	Delete saved design
POST	/uploads/image	Customer +	Upload image to Cloudinary (with MIME + magic byte check)
POST	/uploads/file	Customer +	Upload general file to Cloudinary
DELETE	/uploads/:publicId	Customer +	Delete upload from Cloudinary

7.4 Cart & Addresses — /cart/* /addresses/*

Method	Endpoint	Auth	Description
GET	/cart	Customer	Get current cart with items and totals
POST	/cart/items	Customer	Add item to cart
PUT	/cart/items/:id	Customer	Update item quantity
DELETE	/cart/items/:id	Customer	Remove item from cart
DELETE	/cart/clear	Customer	Clear entire cart

GET	/addresses	Customer	List saved addresses
POST	/addresses	Customer	Add new address
PUT	/addresses/:id	Customer	Update address
DELETE	/addresses/:id	Customer	Delete address
POST	/coupons/validate	Customer	Validate coupon code and return discount

7.5 Orders & Payments — /orders/* /payments/*

Method	Endpoint	Auth	Description
POST	/orders	Customer	Create order from cart (atomic transaction)
GET	/orders	Customer	Paginated order history
GET	/orders/:id	Customer	Order detail with items and customization
POST	/orders/:id/cancel	Customer	Cancel order (before IN_PRODUCTION)
PUT	/orders/:id/status	Admin	Advance order status
PUT	/orders/:id/tracking	Admin	Set tracking number and courier
POST	/payments/create-order	Customer	Create Razorpay order (server-side)
POST	/payments/verify	Customer	Verify payment signature client-side confirm
POST	/payments/webhook	Razorpay	HMAC-verified payment events
POST	/payments/:id/refund	Admin	Initiate refund via Razorpay API
GET	/payments/:orderId	Customer	Get payment status for order

7.6 Notifications — /notifications/*

Method	Endpoint	Auth	Description
GET	/notifications	Customer	Get in-app notification feed (paginated)
PATCH	/notifications/:id/read	Customer	Mark single notification as read
PATCH	/notifications/read-all	Customer	Mark all notifications as read
POST	/admin/notifications/broadcast	SuperAdmin	Broadcast notification to all customers

7.7 Admin — /admin/*

Meth od	Endpoint	Auth	Description
GET	/admin/dashboard	Admin	KPI cards: revenue, orders, top products
GET	/admin/orders	Admin	Filterable order queue with pagination
GET	/admin/customers	Admin	Customer list with order count + status
GET	/admin/analytics	Admin	Revenue and conversion analytics
GET	/admin/reports/export	Admin	Export orders/revenue as CSV or PDF
POST	/coupons	Admin	Create coupon
PUT	/coupons/:id	Admin	Update coupon
DELE TE	/coupons/:id	Admin	Delete coupon
PUT	/reviews/:id	Admin	Approve / reject review
POST	/gallery	Admin	Upload gallery portfolio image
DELE TE	/gallery/:id	Admin	Remove gallery image
PUT	/admin/customers/:id/ status	SuperAdmin	Activate / deactivate customer account
GET	/admin/users	SuperAdmin	List all admins (user management)
POST	/admin/users	SuperAdmin	Create admin account
PUT	/admin/users/:id/role	SuperAdmin	Change admin role

8. 12-Phase Development Plan — 24 Sprints

P1 — Foundation

Sprints: Sprint 1

Duration: 1 week

Objectives

- Establish monorepo with apps/api and apps/web workspaces
- Configure PostgreSQL with Prisma ORM and initial schema (users, addresses, activity_logs)
- Set up Redis (Upstash) for caching, sessions, and job queues
- Configure GitHub Actions CI with quality gates (ESLint, TypeScript, Jest, npm audit)
- Deploy to dev / staging / production environments on Vercel + Railway
- Define shared TypeScript types, AppError class, response envelope, paginator util
- Scaffold Next.js 15 App Router with Tailwind + Shadcn UI
- Configure Axios API client with interceptors; Zustand store skeleton; TanStack Query provider

Acceptance Criteria

- ✓ GET /api/v1/health returns 200 with db:ok and redis:ok in all three environments
- ✓ Prisma migrate deploy runs without errors on staging PostgreSQL
- ✓ GitHub Actions CI completes in under 4 minutes on every PR
- ✓ Next.js app auto-deploys to Vercel preview URL on PR
- ✓ Railway API service deploys from main branch within 5 minutes

P2 — Authentication & RBAC

Sprints: Sprints 2–3

Duration: 2 weeks

Objectives

- User registration with phone OTP verification (Twilio SMS)
- Login with JWT access (15-min) + refresh (7-day) in httpOnly cookies
- Refresh token rotation — SHA-256 hash in Redis, invalidate on use
- Forgot password / reset password via email OTP
- RBAC middleware: CUSTOMER, ADMIN, SUPER_ADMIN
- Admin TOTP 2FA (authenticator app)
- Rate limiting: 5 req/min on auth endpoints; 3 failed OTP → 15-min lockout

Acceptance Criteria

- ✓ Registration → OTP SMS → verify → home with valid session
- ✓ JWT in httpOnly cookie — zero token visible in JS or response JSON
- ✓ Expired access + valid refresh → silent re-auth without user action
- ✓ After logout, refresh token replay returns 401
- ✓ CUSTOMER role returns 403 on /admin/* routes

P3 — Product Management

Sprints: Sprints 4–5

Duration: 2 weeks

Objectives

- Full product CRUD (create / edit / activate / deactivate / soft-delete / restore) — admin only
- Category hierarchy (parent/child) with image and sort order
- Product variants (size, color, material) with individual pricing and stock
- Product image upload via Cloudinary (up to 10 images per product)
- Full-text search using PostgreSQL GIN index (tsvector on name + description)
- Filters: category, price range, rating, availability; Sort: price, newest, best-selling
- Product listing page + detail page on Customer Portal
- Redis caching: products TTL 10min, categories TTL 1hr; invalidation on admin update

Acceptance Criteria

- ✓ Search returns results within 300ms for 1,000-product catalog
- ✓ Admin creates product with 3 variants and 5 images in under 5 minutes
- ✓ isActive=false / is_deleted=true products absent from all customer endpoints
- ✓ Admin activate/deactivate toggle updates storefront within 30 seconds (cache invalidation)
- ✓ Soft-deleted product is restorable from /admin/products with full data intact

P4 — Dynamic Customization Engine

Sprints: Sprints 6–8

Duration: 3 weeks (Critical Path)

Objectives

- Admin no-code field builder UI (drag-drop, 9 field types)
- 9 types: text, textarea, number, image upload, dropdown, checkbox, radio, color picker, dimension
- Per-field validation rules stored in JSONB schema
- Preview template config: canvas x/y position, font, color, image area per field
- FieldRenderer: dynamic input component driven entirely by schema
- Image upload to Cloudinary with MIME whitelist, magic byte check, progress indicator
- Save design to user account for reuse (user_designs table)
- Customization data snapshot stored as JSONB in order_items at checkout

Acceptance Criteria

- ✓ Admin creates 5-field ID Card schema in under 10 minutes
- ✓ FieldRenderer renders all 9 types correctly without code changes
- ✓ Cloudinary upload completes within 3 seconds for 2MB file
- ✓ Schema change after order does not affect historical order_items.customization_data
- ✓ Saved design reloads with all fields pre-populated

P5 — Cart & Checkout

Sprints: Sprints 9–10

Duration: 2 weeks

Objectives

- DB-backed persistent cart (one per user) — survives browser close
- Guest cart in localStorage, migrates to DB on login
- Add / update / remove / clear cart items
- Address management: add, edit, delete, set default
- Coupon validation: code, type, date range, usage limit, minimum order value
- Order summary: subtotal + discount + shipping + total
- Checkout flow: cart review → address → coupon → payment

Acceptance Criteria

- ✓ Cart persists across browser sessions and devices when logged in
- ✓ Same variant twice increments quantity — no duplicate cart item
- ✓ Valid 20% coupon reduces displayed total in real time
- ✓ Checkout blocked with no delivery address selected
- ✓ Out-of-stock items flagged — cannot proceed to checkout

P6 — Payments

Sprints: Sprints 11–12

Duration: 2 weeks

Objectives

- Razorpay server-side order creation with idempotency key
- Razorpay checkout: UPI, card, netbanking, wallets
- HMAC-SHA256 webhook verification on raw body (not parsed JSON)
- Events handled: payment.captured, payment.failed, refund.processed
- Idempotency check: duplicate webhook → 200 no-op
- Stock decrement with row-level lock on capture event only
- Admin-initiated refund via Razorpay API → Bull MQ job

Acceptance Criteria

- ✓ Successful UPI payment: order transitions to PAYMENT_CONFIRMED within 5 seconds
- ✓ Replayed webhook does not double-decrement stock
- ✓ Tampered signature returns 400 — no state change
- ✓ Admin refund → REFUNDED status + customer notification
- ✓ Payment retry up to 3 times without creating new order

P7 — Order Management

Sprints: Sprints 13–14

Duration: 2 weeks

Objectives

- Atomic order creation: cart → order + order_items + payment record in one transaction
- Full status lifecycle: PENDING → PAYMENT_CONFIRMED → IN_PRODUCTION → QUALITY_CHECK → SHIPPED → DELIVERED
- Customer cancellation blocked at IN_PRODUCTION and beyond
- Admin status pipeline: status updater + tracking number entry on SHIPPED
- Order history: paginated, filterable by status
- Order detail: items, customization preview, timeline
- Auto-refund job triggered on CANCELLED status

Acceptance Criteria

- ✓ Order creation is atomic — no partial states possible
- ✓ Customer UI hides cancel button at IN_PRODUCTION; API returns 409
- ✓ Cancellation before production triggers Razorpay refund via Bull MQ
- ✓ Order history loads 20 items/page sorted by created_at DESC in under 200ms

P8 — Admin Dashboard & Management Portal

Sprints: Sprints 15–17

Duration: 3 weeks

Objectives

- Admin dashboard: revenue today/week/month, orders by status, top products, conversion funnel
- Full product management UI: data table, create/edit form, activate/deactivate toggle, soft delete/restore
- Product image and design template upload management in admin form
- Customization field builder (/admin/products/[id]/fields)
- Order queue: filterable by status, bulk status update, tracking number entry
- Customer list with order count, account status, deactivate action
- Coupon CRUD: create percentage/flat/free-shipping coupons
- Review moderation queue (approve/reject/reply)
- Gallery management: upload/delete portfolio images
- CSV/PDF export for orders and revenue reports
- All admin mutations logged to activity_logs

Acceptance Criteria

- ✓ Dashboard KPIs load within 500ms with proper indexes
- ✓ Admin updates order PAYMENT_CONFIRMED → SHIPPED in 3 clicks with tracking saved
- ✓ Export 500 orders as CSV within 5 seconds
- ✓ Field builder changes live on storefront within 30 seconds (cache invalidation)
- ✓ CUSTOMER role attempting /admin/* returns 403

P9 — Notifications

Sprints: Sprints 18–19

Duration: 2 weeks

Objectives

- Bull MQ queues: email-queue (P1), whatsapp-queue (P1), push-queue (P2), db-queue (P3)
- Email via SendGrid: OTP, order confirmed, shipped, delivered, refund HTML templates
- WhatsApp via Twilio: approved template messages for order events
- Push via Firebase FCM: order updates, delivery alerts
- In-app feed: notifications table, bell badge, mark-read
- Retry: 3 attempts exponential back-off per channel
- Dead-letter queue for persistent failures
- Review request auto-sent 3 days after DELIVERED status

Acceptance Criteria

- ✓ Order confirmation email delivered within 30 seconds of payment.captured
- ✓ Failed job retried 3× — persistent failure lands in DLQ without crashing worker
- ✓ In-app bell shows unread count; mark-all-read clears it
- ✓ Notification worker failure does not block order update transaction

P10 — Design Preview Engine

Sprints: Sprints 20–21

Duration: 2 weeks

Objectives

- LivePreview canvas: renders product template + customer field values in real time
- Text injection: font, size, color, position from previewConfig JSONB
- Image injection: uploaded photo placed at configured canvas bounding box
- Background and overlay layer support (product template as base)
- Preview PNG snapshot saved to Cloudinary at checkout → stored in order_items.preview_url
- Admin: download print-ready PNG/PDF per order_item from order detail
- Server-side print export via Puppeteer/Sharp (Bull MQ job)
- Debounced re-render on field input (max 200ms latency)

Acceptance Criteria

- ✓ Text fields update on canvas within 200ms of last keystroke
- ✓ Photo renders within image bounding box — no overflow
- ✓ Preview PNG stored in order_items.preview_url at checkout
- ✓ Admin downloads print-ready PNG from order detail
- ✓ Renders correctly in Chrome, Firefox, Safari

P11 — Testing & Security Audit

Sprints: Sprints 22–23

Duration: 2 weeks

Objectives

- Unit tests: $\geq 80\%$ coverage on all services, utils, validators (Jest)
- Integration tests: all API endpoints against test DB (Supertest)
- E2E tests: register → browse → customize → checkout → confirmation (Playwright)
- Load test: k6, 500 VUs, 10-min sustained, product browse + cart scenario
- Security: OWASP ZAP scan against staging; resolve all high/critical findings
- Dependency audit: npm audit + Snyk; zero high/critical CVEs
- Accessibility: WCAG 2.1 AA — WAVE tool + keyboard navigation audit
- IDOR test: customer A cannot access customer B's orders or cart

Acceptance Criteria

- ✓ LCP $\leq 2.5s$ on product detail page (Lighthouse mobile)
- ✓ API P95 $\leq 300ms$ under 500 concurrent VUs
- ✓ API error rate $\leq 0.1\%$ during load test
- ✓ Zero high/critical OWASP ZAP findings
- ✓ E2E checkout flow ≤ 4 seconds (excluding payment)

P12 — Production Deployment

Sprints: Sprint 24

Duration: 1 week

Objectives

- All Railway production secrets configured
- Razorpay webhook URL registered for production domain
- Cloudinary upload preset configured for production
- SendGrid domain authentication (SPF, DKIM) completed
- Twilio WhatsApp Business templates approved
- Firebase FCM server key in Railway secrets
- Cloudflare DNS + SSL/TLS Full Strict mode
- PgBouncer deployed in front of PostgreSQL
- PostgreSQL read replica provisioned
- Sentry release tracking + Railway alerts + uptime monitoring configured
- Runbook for common incidents written and reviewed

Acceptance Criteria

- ✓ End-to-end smoke test: register → customize ID card → checkout → payment (live) → confirmation email
- ✓ GET /api/v1/health returns 200 with db:ok + redis:ok on production URL
- ✓ Vercel Speed Insights LCP green on first real-user load

- ✓ Sentry test error fires alert to Slack within 2 minutes
- ✓ Railway rollback to previous image in under 5 minutes

9. Security Standards

Requirement	Standard / Implementation
Password hashing	bcrypt cost factor 12 — never store plaintext
Transport encryption	TLS 1.3, HSTS max-age=31536000
JWT signing	HS256, 15-min access, 7-day refresh; secrets rotated quarterly
Refresh token storage	SHA-256 hash in Redis — raw token never persisted
JWT delivery	httpOnly, Secure, SameSite=Strict cookies — never in localStorage or body
File upload validation	MIME whitelist + magic byte verification + 5MB limit + Cloudinary virus scan
SQL injection prevention	Prisma parameterized queries only — no raw SQL in business logic
XSS prevention	CSP headers + Zod input sanitization before any controller
Payment data	PCI-DSS SAQ A only — no card data stored; Razorpay hosted checkout
Webhook security	HMAC-SHA256 on raw body; JSON parser bypassed for webhook endpoint
Rate limiting	Redis sliding window: 100 req/min general; 5/min auth endpoints
RBAC	Middleware on every protected route; 403 (not 404) for unauthorized role
Admin mutations	All logged to activity_logs with userId, action, IP, timestamp
CORS	Explicit allowlist; credentials: same-origin only
Edge security	Cloudflare WAF + OWASP Top 10 rules + DDoS protection + Bot Management

10. Non-Functional Requirements

10.1 Performance Targets

Metric	Target	Measurement
Largest Contentful Paint (LCP)	< 2.5s	Vercel Speed Insights
First Input Delay (FID)	< 100ms	Core Web Vitals
API P95 response time	< 300ms	Railway Metrics
API P99 response time	< 800ms	Railway Metrics
Database query P95	< 50ms	pg_stat_statements
File upload processing	< 3s for ≤ 5MB	Cloudinary API
Customization preview render	< 200ms	Client-side debounce

10.2 Availability & Reliability

Metric	Target
Uptime SLA	99.9% (< 8.7 hrs downtime/year)
RTO (Recovery Time Objective)	< 1 hour
RPO (Recovery Point Objective)	< 15 minutes
Database backup	Daily full + continuous WAL archiving
Deployment rollback	< 5 minutes via Vercel/Railway

10.3 Scalability

Dimension	Target
Concurrent users	10,000+
Peak orders/hour	500
File uploads/hour	2,000
Database connections (pooled)	100 max via PgBouncer
Storage growth tolerance	100 GB/year via Cloudinary

11. Monitoring & Alerting

Alert	Threshold	Severity	Channel
API error rate	> 1% over 5 min	P1	Slack + Email
API P99 latency	> 2000ms	P1	Slack
DB connection pool saturation	> 90%	P0	Slack + PagerDuty
Payment webhook failures	Any	P0	Slack + PagerDuty
Failed deploys	Any	P1	Slack
Disk usage	> 80%	P2	Email
Uptime check failure	2 consecutive fails	P0	All channels
CPU usage (Railway pod)	> 80% for 5 min	P1	Slack
Memory usage	> 85%	P1	Slack
Cloudflare origin error rate	> 1%	P1	Slack

12. Architecture Reconstruction Summary

Item	Count
Total Development Phases	12
Total Sprints (1-week each)	24
Total API Endpoints	62
Total Database Tables	22
Total Features Documented	89
Total Workflows Documented	12 (one per phase)
Total Roles	4 (Guest, Customer, Admin, SuperAdmin)
Total Notification Channels	4 (Email, WhatsApp, Push, In-App)
Total Field Types (Customization)	9
Total Order Status States	8
Total Payment Status States	5

Merko architecture is fully reconstructed and ready for Cursor implementation.